

FleetMan

Plateforme de Gestion de Flotte Intelligente

Rapport d'Évolution du Backend

Architecture Hexagonale Réactive — Analyse de l'Existant et des Apports

Projet : **TraEnSys — Module Fleet Management**

Technologie : **Spring Boot 3.2 · WebFlux · R2DBC · PostgreSQL**

Architecture : **Hexagonale Réactive (Ports & Adapters)**

Version : **0.0.1-SNAPSHOT**

Date : **Mai 2026**

Niveau : **5^{eme} annee Genie Informatique**

Résumé Exécutif

Ce rapport présente une analyse détaillée de l'architecture et de l'évolution du backend de la plateforme **FleetMan**, un système de gestion de flotte de véhicules développé dans le cadre d'un projet de fin d'études de 5^{ème} année Génie Informatique.

Le document est structuré en deux grandes parties :

- **Partie I — L'Existant** : Description exhaustive de l'architecture hexagonale réactive mise en place initialement, couvrant les modules d'authentification, de gestion des véhicules, des conducteurs, des flottes, du géorepérage et des trajets. Cette partie détaille les choix technologiques (Spring Boot 3.2, WebFlux, R2DBC, PostgreSQL, Redis, Kafka), les modèles de domaine, les ports et adapters, ainsi que la configuration de sécurité JWT.
- **Partie II — Les Apports** : Description précise du module *Opérations Terrain* intégré au projet existant, comprenant trois nouveaux agrégats métier (Maintenance, Incident, Recharge Carburant), leurs modèles de domaine purs, leurs use cases réactifs, leurs adapters de persistance R2DBC, leurs migrations Liquibase et leurs controllers REST documentés Swagger.

L'ensemble du système respecte scrupuleusement les principes de l'architecture hexagonale (Ports & Adapters), du Domain-Driven Design (DDD) et de la programmation réactive (Project Reactor), garantissant une séparation stricte des responsabilités et une testabilité maximale.

Résultat Global

Le backend FleetMan expose **plus de 80 endpoints REST** documentés via Swagger/OpenAPI, gère **14 modules fonctionnels** distincts, et s'appuie sur **10 migrations Liquibase** pour une base de données PostgreSQL structurée en schéma **fleet**.

Table des matières

french

Résumé Exécutif	1
Introduction Générale	9
I L'Architecture Existante du Backend FleetMan	11
1 Fondements Technologiques et Choix d'Architecture	12
1.1 Présentation Générale du Projet	12
1.2 Stack Technologique	12
1.2.1 Technologies Principales	12
1.2.2 Justification des Choix Technologiques	13
2 L'Architecture Hexagonale Réactive	15
2.1 Présentation du Pattern Hexagonal	15
2.2 Structure des Packages	15
2.3 La Règle de Dépendance	16
2.4 Les Ports : Contrats du Domaine	17
2.4.1 Ports Entrants (Use Cases)	17
2.4.2 Ports Sortants	17
3 Les Modules Métier Existants	19
3.1 Les Acteurs du Système	19
3.2 Module d'Authentification	19
3.2.1 Architecture du Module Auth	19
3.2.2 Sécurité JWT Réactive	20
3.2.3 Configuration de Sécurité	20
3.2.4 CORS	21
3.3 Module de Gestion des Véhicules	21
3.3.1 Modèle de Domaine	21
3.3.2 Paramètres du Véhicule	22
3.3.3 Référentiels Véhicule	22
3.3.4 Endpoints REST du Module Véhicule	23
3.4 Module de Gestion des Conducteurs	23

3.4.1	Modèle de Domaine	23
3.4.2	Fonctionnalités du Module Conducteur	24
3.4.3	Endpoints REST du Module Conducteur	24
3.5	Module de Gestion des Flottes	24
3.5.1	Modèle de Domaine	24
3.5.2	Fonctionnalités du Module Flotte	25
3.6	Module de Gestion des Trajets	25
3.6.1	Modèle de Domaine	25
3.6.2	Cycle de Vie d'un Trajet	25
3.6.3	Télémétrie GPS	26
3.6.4	Endpoints REST du Module Trajet	26
3.7	Module de Géorepérage (Geofencing)	26
3.7.1	Présentation	26
3.7.2	Architecture du Module Geofence	27
3.7.3	Modèles de Domaine	27
3.7.4	Endpoints REST du Module Geofence	27
3.8	Module de Notifications	28
3.9	Module d'Administration	29
3.9.1	Administrateur	29
3.9.2	Super Administrateur	29
3.9.3	Initialisation Automatique	29
4	La Couche Infrastructure Existante	30
4.1	La Base de Données PostgreSQL	30
4.1.1	Organisation en Schémas	30
4.1.2	Migrations Liquibase	30
4.1.3	Schéma de la Base de Données Principale	31
4.2	Les Adapters de Persistance R2DBC	31
4.2.1	Pattern Adapter de Persistance	31
4.2.2	Repositories Spring Data R2DBC	32
4.3	Les Adapters Externes (HTTP Clients)	32
4.3.1	Architecture des Clients HTTP	32
4.3.2	Adapters Externes	33
4.4	Les Adapters Kafka	33
4.4.1	Configuration Kafka	33
4.4.2	Producteurs et Consommateurs	34
4.5	Les Mappers MapStruct	34
4.6	Configuration du Monitoring	34
4.6.1	Spring Actuator	34

4.6.2	Métriques Prometheus	34
4.6.3	Circuit Breaker Resilience4j	35
5	Configuration Système et Déploiement	36
5.1	Configuration de l'Application	36
5.1.1	Profils Spring	36
5.1.2	Configuration des Services Externes	36
5.2	Containerisation Docker	36
5.2.1	Dockerfile	37
5.2.2	Docker Compose	37
5.3	CI/CD	37
5.4	Documentation API	37
II	Les Apports : Module Opérations Terrain	38
6	Analyse des Besoins et Conception du Module Opérations	39
6.1	Contexte et Motivation	39
6.2	Analyse des Besoins Fonctionnels	39
6.2.1	Besoins liés aux Maintenances	39
6.2.2	Besoins liés aux Incidents	40
6.2.3	Besoins liés aux Recharges Carburant	40
6.3	Analyse des Besoins Non-Fonctionnels	40
6.4	Identification des Agrégats DDD	41
6.5	Stratégie d'Intégration	41
7	Implémentation — Couche Domaine	42
7.1	Le Value Object <code>Coordinates</code>	42
7.1.1	Conception	42
7.1.2	Caractéristiques Techniques	42
7.2	L'Entité <code>Maintenance</code>	43
7.2.1	Conception	43
7.2.2	Invariants Métier	43
7.2.3	Méthodes Métier	43
7.2.4	Données Dénormalisées	44
7.3	L'Entité <code>Incident</code> — Machine à États	44
7.3.1	Conception	44
7.3.2	Enums du Domaine	44
7.3.3	Machine à États	45
7.3.4	Méthodes Métier	45

7.4	L'Entité <code>FuelRecharge</code>	45
7.4.1	Conception	46
7.4.2	Méthode Métier <code>unitCost()</code>	46
7.4.3	Enum <code>StationName</code>	46
7.5	L'Exception Métier <code>OperationException</code>	46
7.5.1	Conception	46
8	Implémentation — Ports et Services Applicatifs	48
8.1	Les Ports Entrants (Use Cases)	48
8.1.1	<code>ManageMaintenanceUseCase</code>	48
8.1.2	<code>ManageIncidentUseCase</code>	49
8.1.3	<code>ManageFuelRechargeUseCase</code>	49
8.2	Les Ports Sortants	49
8.2.1	<code>OperationEventPort</code> — Port Événementiel	49
8.3	Les Services Applicatifs	50
8.3.1	<code>MaintenanceService</code>	50
8.3.2	Synchronisation Inter-Modules	51
8.3.3	<code>IncidentService</code> — Gestion du Cycle de Vie	51
8.3.4	<code>FuelRechargeService</code> — Synchronisation du Niveau Carburant	52
9	Implémentation — Couche Infrastructure	54
9.1	Migration Liquibase — Changeset 010	54
9.1.1	Table <code>fleet.maintenances</code>	54
9.1.2	Table <code>fleet.incidents</code>	55
9.1.3	Table <code>fleet.fuel_recharges</code>	55
9.1.4	Décisions de Conception des Tables	56
9.2	Les Entités R2DBC	56
9.2.1	Pattern des Entités	56
9.3	Les Repositories R2DBC	57
9.3.1	<code>IncidentR2dbcRepository</code> — Le Plus Riche	57
9.4	Les Adapters de Persistance	58
9.4.1	Conversion Entity ↔ Domain	58
9.5	L'Adapter Kafka — <code>KafkaOperationEventAdapter</code>	59
10	Implémentation — API REST et Résultats	61
10.1	Les DTOs de Requête et Réponse	61
10.1.1	DTOs de Requête	61
10.1.2	DTOs de Réponse	62
10.2	Les Controllers WebFlux	62
10.2.1	<code>MaintenanceController</code>	62

10.2.2	Récapitulatif des Endpoints du Module Opérations	63
10.3	Bilan des Fichiers Créés	65
10.3.1	Récapitulatif Global	65
10.3.2	Fichiers Modifiés	66
11	Décisions Techniques et Bilan	67
11.1	Décisions Techniques Majeures	67
11.1.1	Pas de Records pour les Entités Mutables	67
11.1.2	Fire & Forget pour les Événements	67
11.1.3	Synchronisation Inter-Modules via Repositories Directs	68
11.1.4	Enums en VARCHAR	68
11.1.5	Coordonnées en Deux Colonnes	68
11.2	Respect des Règles Architecturales	68
11.3	Bilan Quantitatif	69
11.4	Intégration avec les Modules Existants	69
	Conclusion Générale	70
A	Glossaire Technique	72
B	Récapitulatif des Endpoints REST	74

Liste des tableaux

french

1.1	Stack technologique complète du projet FleetMan	13
2.1	Ports entrants (Use Cases) du projet FleetMan	17
2.2	Ports sortants du projet FleetMan	18
3.1	Acteurs et rôles du système FleetMan	19
3.2	Groupes de paramètres du véhicule	22
3.3	Endpoints REST du module Véhicule	23
3.4	Endpoints REST du module Conducteur	24
3.5	Endpoints REST du module Trajet	26
3.6	Endpoints REST du module Geofence	28
4.1	Historique des migrations Liquibase	30
4.2	Tables du schéma fleet avant les ajouts	31
4.3	Clients HTTP externes du projet	33
6.1	Agrégats DDD du module Opérations Terrain	41
6.2	Adaptations réalisées lors de l'intégration	41
7.1	Méthodes métier de l'entité Maintenance	44
7.2	Codes d'erreur de OperationException	47
10.1	Endpoints REST du module Maintenance	64
10.2	Endpoints REST du module Incident	64
10.3	Endpoints REST du module Recharge Carburant	64
10.4	Récapitulatif des 37 fichiers créés	66
10.5	Fichiers modifiés lors de l'intégration	66
11.1	Vérification du respect des règles architecturales	69
11.2	Bilan quantitatif des apports	69
A.1	Glossaire des termes techniques	73
B.1	Sélection des endpoints REST principaux (les modules en gras sont les apports)	75

Table des figures

french

2.1	Architecture hexagonale du projet FleetMan	15
7.1	Machine à états de l'entité Incident	45

Introduction Générale

Contexte du Projet

Dans un contexte économique où la maîtrise des coûts logistiques est devenue un enjeu stratégique majeur pour les entreprises de transport en Afrique centrale, la gestion efficace d'une flotte de véhicules représente un défi opérationnel quotidien. Les entreprises de taxis, de livraison et de transport de marchandises font face à des problématiques récurrentes : manque de visibilité en temps réel sur la position de leurs véhicules, absence de traçabilité des incidents et des maintenances, difficulté à contrôler la consommation de carburant, et impossibilité de définir des zones géographiques d'activité autorisées.

C'est dans ce contexte que le projet **FleetMan** (anciennement *TraEnSys — Module Fleet Management*) a été conçu : une plateforme B2B permettant aux gestionnaires de flotte de superviser leurs véhicules en temps réel, de gérer leurs conducteurs, et de contrôler le respect des zones géographiques d'activité via le géorepérage.

Objectifs du Rapport

Ce rapport d'évolution a pour objectifs de :

1. Documenter de manière exhaustive l'architecture technique du backend existant, en explicitant les choix de conception et les patterns architecturaux adoptés.
2. Présenter en détail les modules fonctionnels initialement développés, leurs modèles de données, leurs interfaces de programmation et leurs interactions.
3. Décrire précisément les apports réalisés dans le cadre de ce projet de fin d'études, notamment l'intégration du module *Opérations Terrain* qui enrichit significativement les capacités de la plateforme.
4. Justifier les décisions techniques prises lors de l'intégration, en montrant comment elles s'inscrivent dans la cohérence architecturale du projet.

Organisation du Document

Le présent rapport est organisé en deux parties principales :

- **Partie I** couvre l'architecture existante du backend FleetMan avant nos contributions. Elle comprend cinq chapitres traitant respectivement des fondements technologiques, de l'architecture hexagonale, des modules métier de base, de la couche infrastructure, et de la configuration système.
- **Partie II** présente les apports réalisés dans le cadre de ce projet. Elle comprend quatre chapitres décrivant l'analyse des besoins, la conception du module Opérations Terrain, son implémentation technique complète, et les résultats obtenus.

Première partie

L'Architecture Existante du Backend FleetMan

Chapitre 1

Fondements Technologiques et Choix d'Architecture

1.1 Présentation Générale du Projet

FleetMan est un service backend réactif développé en Java 21 avec le framework Spring Boot 3.2.6. Il constitue le cœur applicatif d'une plateforme de gestion de flotte de véhicules destinée aux entreprises de transport. Le projet est identifié sous les coordonnées Maven suivantes :

Attribut	Valeur
groupId	com.yowjob
artifactId	reactive-hexagonal
version	0.0.1-SNAPSHOT
Java	21 (LTS)
Spring Boot	3.2.6
Spring Cloud	2023.0.0

1.2 Stack Technologique

1.2.1 Technologies Principales

Le tableau suivant récapitule l'ensemble des technologies utilisées dans le projet :

Technologie	Version	Rôle
Spring Boot WebFlux	3.2.6	Framework web réactif (non-bloquant)
Spring Data R2DBC	3.2.6	Accès base de données réactif
Spring Security	6.x	Sécurité, JWT, RBAC
Spring Kafka	3.x	Messagerie asynchrone
Reactor Kafka	1.3.22	Kafka réactif (Mono/Flux)

Technologie	Version	Rôle
PostgreSQL (R2DBC)	r2dbc-postgresql	Driver réactif PostgreSQL
PostgreSQL (JDBC)	runtime	Driver JDBC pour Liquibase
Redis (Réactif)	Spring Data Redis	Cache télémétrie temps réel
Liquibase	4.x	Migrations de base de données
MapStruct	1.5.5.Final	Mapping objet-objet généré
Lombok	1.18.30	Réduction du code boilerplate
SpringDoc OpenAPI	2.5.0	Documentation Swagger/OpenAPI 3
Resilience4j	Spring Cloud	Circuit Breaker réactif
Micrometer theus	Prometheus	Métriques et monitoring
Spring Actuator	3.2.6	Endpoints de santé et métriques

TAB. 1.1 : Stack technologique complète du projet Fleet-Man

1.2.2 Justification des Choix Technologiques

Spring WebFlux et la Programmation Réactive

Le choix de Spring WebFlux plutôt que Spring MVC (bloquant) est fondamental dans ce projet. La gestion de flotte implique des flux de données continus : remontées GPS des véhicules, alertes de géorepérage, notifications en temps réel. Un modèle de programmation réactif basé sur Project Reactor (`Mono<T>` et `Flux<T>`) permet de traiter ces flux de manière non-bloquante, avec une utilisation optimale des ressources système (un seul thread peut gérer des milliers de connexions simultanées).

Dans ce projet, **toutes** les méthodes de service, de port et de repository retournent soit un `Mono<T>` (0 ou 1 résultat), soit un `Flux<T>` (0 à N résultats). Aucun appel bloquant n'est autorisé dans la chaîne de traitement principale.

R2DBC vs JPA/Hibernate

Spring Data R2DBC est utilisé à la place de JPA/Hibernate pour l'accès à la base de données. Ce choix est cohérent avec le paradigme réactif : JPA est fondamentalement bloquant (il utilise JDBC en interne), ce qui serait incompatible avec WebFlux. R2DBC fournit un accès réactif natif à PostgreSQL, permettant des requêtes non-bloquantes.

Liquibase pour les Migrations

Liquibase est utilisé pour gérer les migrations de schéma de base de données. Il nécessite une connexion JDBC (bloquante), c'est pourquoi le projet maintient deux configurations de connexion distinctes : R2DBC pour l'application en runtime, et JDBC uniquement pour Liquibase au démarrage.

Redis pour la Télémétrie

Redis est utilisé comme cache en mémoire pour stocker les données de télémétrie des véhicules (position GPS, vitesse, niveau de carburant). Ces données sont hautement volatiles et nécessitent des accès en lecture/écriture très rapides. Redis, avec son accès en mémoire, est parfaitement adapté à ce cas d'usage.

Apache Kafka pour la Messagerie

Kafka est utilisé pour la communication asynchrone entre modules, notamment pour la publication d'événements de géorepérage et d'opérations terrain. Ce découplage permet à chaque module de fonctionner indépendamment et garantit la résilience du système en cas de défaillance partielle.

Chapitre 2

L'Architecture Hexagonale Réactive

2.1 Présentation du Pattern Hexagonal

L'architecture hexagonale, également connue sous le nom d'*Architecture Ports & Adapters* (Alistair Cockburn, 2005), est le pattern architectural central du projet FleetMan. Son principe fondamental est d'isoler le cœur métier (le domaine) de toutes les dépendances techniques (base de données, API externes, frameworks).

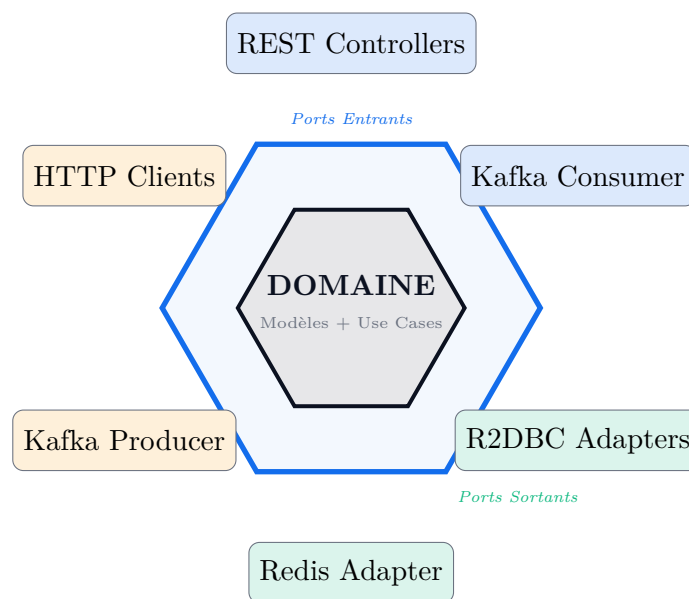


FIG. 2.1 : Architecture hexagonale du projet FleetMan

2.2 Structure des Packages

Le projet est organisé selon trois couches principales, reflétant directement l'architecture hexagonale :

Listing 2.1 : Structure des packages Java

```
com.yowjob.fleet/  
  FleetManagementApplication.java    # Point d'entrée Spring Boot
```



```
domain/                                # COUCHE DOMAINE (pur Java)
  model/                               # Entités et Value Objects
  ports/
    in/                                # Use Cases (ports entrants)
    out/                               # Ports sortants (persistance,
      events)                          # Exceptions métier typées
  exception/                           #

application/                           # COUCHE APPLICATION
  service/                             # Implémentations des Use Cases

infrastructure/                         # COUCHE INFRASTRUCTURE
  adapters/
    inbound/
      rest/                            # Controllers WebFlux
      dto/                             # DTOs de requête/réponse
      messaging/                       # Consommateurs Kafka
    outbound/
      persistence/                    # Adapters R2DBC
      entity/                         # Entités R2DBC
      repository/                     # Repositories Spring Data
      external/                       # Clients HTTP externes
      client/                         # WebClient wrappers
      dto/                            # DTOs externes
      messaging/                      # Producteurs Kafka
  config/                              # Configurations Spring
  security/                            # JWT, CORS
  bootstrap/                          # Initialisation données
  mappers/                            # MapStruct mappers
```

2.3 La Règle de Dépendance

La règle fondamentale de l'architecture hexagonale est que **les dépendances ne pointent que vers l'intérieur**. Concrètement dans ce projet :

- Le package `domain` ne contient **aucune** annotation Spring (`@Service`, `@Repository`), JPA (`@Entity`, `@Column`), ou Jackson (`@JsonProperty`).
- Les classes du domaine sont du **Java pur** : records immuables, classes avec méthodes métier, interfaces.
- Les services applicatifs (`application/service`) dépendent uniquement des ports définis dans `domain/ports`.
- Les adapters d'infrastructure (`infrastructure/adapters`) implémentent

les ports et dépendent des frameworks techniques.

2.4 Les Ports : Contrats du Domaine

2.4.1 Ports Entrants (Use Cases)

Les ports entrants définissent ce que le domaine *offre* au monde extérieur. Chaque port est une interface Java dont les méthodes retournent des types réactifs :

Interface (Port Entrant)	Responsabilité
AuthUseCase	Authentification, inscription, rafraîchissement de token
ManageVehicleUseCase	CRUD véhicules, paramètres, télémétrie
ManageDriverUseCase	CRUD conducteurs, assignation véhicule
ManageFleetUseCase	CRUD flottes, assignation véhicules
ManageFleetManagerUseCase	Gestion des gestionnaires de flotte
ManageTripUseCase	Démarrage/fin de trajet, télémétrie GPS
ManageGeofenceUseCase	Zones géographiques, alertes
ManageIncidentUseCase	Incidents terrain, cycle de vie
ManageMaintenanceUseCase	Maintenances véhicules
ManageFuelRechargeUseCase	Recharges carburant
ManageAdminUseCase	Administration des comptes
ManageSuperAdminUseCase	Super-administration
AdminStatsUseCase	Statistiques globales

TAB. 2.1 : Ports entrants (Use Cases) du projet FleetMan

2.4.2 Ports Sortants

Les ports sortants définissent ce que le domaine *requiert* de l'infrastructure :

Interface (Port Sortant)	Responsabilité
AuthPort	Délégation de l'authentification au service Auth externe
VehiclePersistencePort	Persistance des véhicules
DriverPersistencePort	Persistance des conducteurs
FleetRepositoryPort	Persistance des flottes
FleetManagerPersistencePort	Persistance des gestionnaires
MaintenancePersistencePort	Persistance des maintenances

Interface (Port Sortant)	Responsabilité
IncidentPersistencePort	Persistance des incidents
FuelRechargePersistencePort	Persistance des recharges carburant
GeofencePersistencePort	Persistance des zones géographiques
IncidentPersistencePort	Persistance des incidents
ExternalGeofencePort	Communication avec le service géorepérage
ExternalVehiclePort	Communication avec le service véhicule externe
ExternalPaymentPort	Communication avec le service paiement
OperationEventPort	Publication d'événements opérationnels (Kafka)
SendNotificationPort	Envoi de notifications
StatisticsPort	Calcul de statistiques
DistanceCalculatorPort	Calcul de distances GPS
NotificationHistoryRepositoryPort	Historique des notifications
NotificationSettingsRepositoryPort	Préférences de notification

TAB. 2.2 : Ports sortants du projet FleetMan

Chapitre 3

Les Modules Métier Existants

3.1 Les Acteurs du Système

Le système FleetMan distingue trois types d'acteurs, chacun disposant de rôles et de permissions spécifiques :

Acteur	Rôle Spring	Responsabilités
Super Administrateur	SUPER_ADMIN	Gestion globale de la plateforme, création des administrateurs
Administrateur	FLEET_ADMIN	Gestion des gestionnaires de flotte, supervision
Gestionnaire de Flotte	FLEET_MANAGER	Gestion de sa flotte, véhicules, conducteurs, zones
Conducteur	FLEET_DRIVER	Démarrage/fin de trajet, déclaration d'incidents

TAB. 3.1 : Acteurs et rôles du système FleetMan

3.2 Module d'Authentification

3.2.1 Architecture du Module Auth

L'authentification dans FleetMan est déléguée à un service externe centralisé (service Auth de l'écosystème TraEnSys). Le module local ne gère pas les mots de passe : il délègue toutes les opérations d'authentification via le port `AuthPort`.

Listing 3.1 : Interface AuthUseCase — Port entrant d'authentification

```
1 public interface AuthUseCase {
2     Mono<AuthPort.AuthResponse> login(String identifiant, String
        password);
3     Mono<AuthPort.AuthResponse> refreshToken(String refreshToken);
4     Mono<AuthPort.AuthResponse> register(RegisterCommand command);
```

```
5
6 record RegisterCommand(
7     String username, String password, String email,
8     String phone, String firstName, String lastName,
9     List<String> roles, FileContent photo
10 ) {}
11
12 record FileContent(String filename, String contentType, byte[] data
13 ) {}
14 }
```

3.2.2 Sécurité JWT Réactive

La sécurité est implémentée via deux composants clés dans `infrastructure/config/security/` :

- **BearerTokenServerAuthenticationConverter** : Extraît le token JWT de l'en-tête `Authorization: Bearer <token>` de chaque requête HTTP entrante.
- **JwtAuthenticationManager** : Valide le token JWT en interrogeant le service Auth externe. En cas de succès, construit un objet `Authentication` contenant les informations de l'utilisateur (`UserDetail` avec `id`, `username`, `roles`).

3.2.3 Configuration de Sécurité

La classe `SecurityConfig` configure la chaîne de filtres WebFlux Security :

Listing 3.2 : Configuration de sécurité WebFlux

```
1 @Configuration
2 @EnableWebFluxSecurity
3 @EnableReactiveMethodSecurity
4 public class SecurityConfig {
5
6     @Bean
7     public SecurityWebFilterChain springSecurityFilterChain(
8         ServerHttpSecurity http) {
9         return http
10             .cors(cors -> cors.configurationSource(
11                 corsConfigurationSource()))
12             .csrf(ServerHttpSecurity.CsrfSpec::disable)
13             .authorizeExchange(exchanges -> exchanges
14                 .pathMatchers("/v3/api-docs/**", "/swagger-ui/**",
15                     "/actuator/**", "/api/v1/auth/**").
16                     permitAll())
17     }
```

```
14         .pathMatchers(HttpMethod.OPTIONS).permitAll()  
15         .anyExchange().authenticated()  
16     )  
17     .addFilterAt(jwtFilter, SecurityWebFiltersOrder.  
18         AUTHENTICATION)  
19     .build();  
20 }
```

Les routes publiques incluent la documentation Swagger, les endpoints Actuator et les routes d'authentification. Toutes les autres routes nécessitent un token JWT valide. Le contrôle d'accès fin est assuré par `@PreAuthorize` sur chaque méthode de controller.

3.2.4 CORS

La configuration CORS autorise toutes les origines (`allowedOriginPatterns: "*"`) avec toutes les méthodes HTTP et tous les en-têtes, ce qui est adapté à un environnement de développement multi-plateforme (web, mobile Android).

3.3 Module de Gestion des Véhicules

3.3.1 Modèle de Domaine

Le véhicule est modélisé comme un **record Java immuable** dans le domaine :

Listing 3.3 : Modèle de domaine Vehicle

```
1 public record Vehicle(  
2     UUID id,  
3     UUID fleetId,           // Flotte d'appartenance (optionnelle)  
4     UUID managerId,        // Gestionnaire propriétaire (obligatoire)  
5     UUID currentDriverId,   // Conducteur actuellement assigné  
6     UUID vehicleTypeId,    // Type de véhicule (référentiel)  
7     String licensePlate,    // Plaque d'immatriculation (unique)  
8     String vehicleSerialNumber,  
9     String brand, String model,  
10    Integer manufacturingYear,  
11    String transmissionType, String fuelType,  
12    Double tankCapacity,  
13    Integer totalSeatNumber,  
14    Double averageFuelConsumption,  
15    String color, String status,  
16    String photoUrl, String serialNumberPhotoUrl,  
17    String registrationPhotoUrl,  
18    List<String> illustrationImages,
```

```

19 VehicleParameters.Financial financialParameters,
20 VehicleParameters.Maintenance maintenanceParameters,
21 VehicleParameters.Operational operationalParameters,
22 String geofenceRemoteId
23 ) {}

```

3.3.2 Paramètres du Véhicule

Le véhicule est enrichi de trois groupes de paramètres, modélisés comme des records imbriqués dans `VehicleParameters` :

Groupe	Champs
Financial	insuranceNumber, insuranceExpiredAt, registeredAt, purchasedAt, depreciationRate, costPerKm
Maintenance	lastMaintenanceAt, nextMaintenanceDue, engineStatus (OK/- NEEDS_SERVICE/OUT_OF_SERVICE), batteryHealth, maintenanceStatus (UP_TO_DATE/PENDING/OVERDUE)
Operational	status, currentSpeed, fuelLevel, mileage, odometerReading, bearing, timestamp

TAB. 3.2 : Groupes de paramètres du véhicule

3.3.3 Référentiels Véhicule

Le module véhicule gère également un ensemble de référentiels (données de base) stockés localement :

- **Type de véhicule** (`vehicle_types`) : BUS, CAR, TRUCK, MOTOR-CYCLE, etc.
- **Fabricants** (`manufacturers`) : constructeurs industriels
- **Marques** (`brands`) : marques commerciales
- **Modèles** (`models`) : modèles de véhicules
- **Tailles** (`sizes`) : gabarits
- **Types d'usage** (`usages`) : Taxi, VIP, Livraison, etc.

- **Types de carburant** (fuel_types) : Essence, Diesel, Électrique, etc.
- **Types de transmission** (transmissions) : Manuelle, Automatique, etc.
- **Couleurs** (colors) : catalogue des couleurs autorisées

3.3.4 Endpoints REST du Module Véhicule

Méthode	URL	Rôle requis	Description
POST	/api/v1/vehicles	MANAGER	Créer un véhicule
GET	/api/v1/vehicles	MANAGER	Lister mes véhicules
GET	/api/v1/vehicles/{id}	MANAGER/ADMIN	Détails complets
PATCH	/api/v1/vehicles/{id}	MANAGER	Mise à jour partielle
PUT	/api/v1/vehicles/{id}/financial-parameters	MANAGER	Paramètres financiers
PUT	/api/v1/vehicles/{id}/maintenance-parameters	MANAGER	Paramètres maintenance
GET	/api/v1/vehicles/{id}/operational-data	DRIVER/MANAGER	Rétrie temps réel
PATCH	/api/v1/vehicles/{id}/operational-data	DRIVER	Mise à jour terrain
DELETE	/api/v1/vehicles/{id}	MANAGER	Supprimer un véhicule
GET	/api/v1/vehicles/lookup/{resource}	PUBLIC	Référentiels
GET	/api/v1/vehicles/resources/all	PUBLIC	Catalogue complet

TAB. 3.3 : Endpoints REST du module Véhicule

3.4 Module de Gestion des Conducteurs

3.4.1 Modèle de Domaine

Le conducteur est modélisé comme un record Java immuable :

Listing 3.4 : Modèle de domaine Driver


```

1 public record Driver(
2     UUID userId,           // ID partagé avec le service Auth
3     UUID fleetId,          // Flotte d'appartenance
4     String licenceNumber,  // Numéro de permis (unique)
5     String status,         // ACTIVE / INACTIVE
6     UUID assignedVehicleId, // Véhicule actuellement assigné
7     String photoUrl        // URL de la photo de profil
8 ) {}

```

3.4.2 Fonctionnalités du Module Conducteur

Le module conducteur gère les opérations suivantes :

- **Enregistrement** : Création du compte Auth + profil local + photo de profil (upload multipart)
- **Listing** : Filtrage par flotte et par état d'assignation
- **Recherche** : Par email ou username
- **Assignation** : Association d'un conducteur à un véhicule
- **Désassignation** : Libération du conducteur de son véhicule

3.4.3 Endpoints REST du Module Conducteur

Méthode	URL	Rôle requis	Description
POST	/api/v1/fleets/{fleetId}/drivers/register	MANAGER	Créer un conducteur
GET	/api/v1/drivers	Authentifié	Lister les conducteurs
GET	/api/v1/drivers/search	Authentifié	Rechercher par identifiant
POST	/api/v1/drivers/{userId}/assign-vehicle	MANAGER	Assigner un véhicule
POST	/api/v1/drivers/{userId}/unassign-vehicle	MANAGER	Désassigner

TAB. 3.4 : Endpoints REST du module Conducteur

3.5 Module de Gestion des Flottes

3.5.1 Modèle de Domaine

Listing 3.5 : Modèle de domaine Fleet

```
1 public record Fleet(  
2     UUID id,  
3     UUID managerId,      // Gestionnaire propriétaire  
4     String name,         // Nom de la flotte  
5     String phoneNumber,  // Numéro de contact  
6     Instant createdAt,   // Date de création  
7     Integer vehicleCount // Nombre de véhicules (calculé)  
8 ) {}
```

3.5.2 Fonctionnalités du Module Flotte

Le module flotte permet au gestionnaire de :

- Créer et gérer plusieurs flottes (une entreprise peut avoir plusieurs flottes distinctes)
- Assigner des véhicules à une flotte
- Consulter les statistiques par flotte
- Gérer les zones de géorepérage associées à une flotte

3.6 Module de Gestion des Trajets

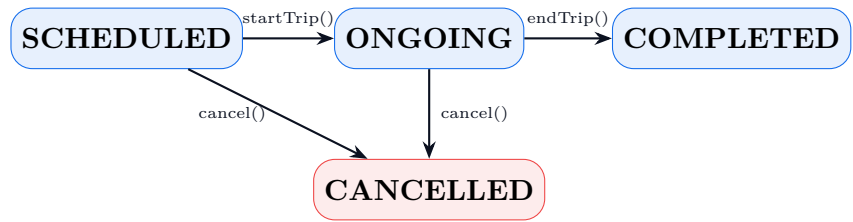
3.6.1 Modèle de Domaine

Listing 3.6 : Modèle de domaine Trip

```
1 public record Trip(  
2     UUID id,  
3     UUID vehicleId,  
4     UUID driverId,  
5     String status,      // SCHEDULED, ONGOING, COMPLETED, CANCELLED  
6     LocalDate startDate,  
7     LocalTime startTime,  
8     LocalDate endDate,  
9     LocalTime endTime,  
10    Double distanceKm,  
11    Integer durationMinutes  
12 ) {}
```

3.6.2 Cycle de Vie d'un Trajet

Un trajet suit un cycle de vie précis :



3.6.3 Télémétrie GPS

Pendant un trajet en cours (ONGOING), le conducteur envoie régulièrement des données de télémétrie via l'endpoint `POST /api/v1/trips/{id}/telemetry`. Ces données (latitude, longitude, vitesse) sont stockées dans Redis pour un accès temps réel, et déclenchent la vérification des règles de géorepérage.

3.6.4 Endpoints REST du Module Trajet

Méthode URL		Rôle requis	Description
POST	/api/v1/trips/start	DRIVER	Démarrer un trajet
POST	/api/v1/trips/{id}/telemetry	DRIVER	Envoyer position GPS
POST	/api/v1/trips/{id}/end	DRIVER	Terminer un trajet
GET	/api/v1/trips/my-active	DRIVER	Mon trajet en cours
GET	/api/v1/trips/{id}	MANAGER	Détails d'un trajet
GET	/api/v1/trips	MANAGER	Lister les trajets

TAB. 3.5 : Endpoints REST du module Trajet

3.7 Module de Géorepérage (Geofencing)

3.7.1 Présentation

Le géorepérage est l'une des fonctionnalités clés de FleetMan. Il permet aux gestionnaires de définir des zones géographiques (polygones ou cercles) et de recevoir des

alertes lorsqu'un véhicule entre ou sort de ces zones.

3.7.2 Architecture du Module Geofence

Le module géorepérage est hybride : il stocke les métadonnées localement (en base PostgreSQL) mais délègue le moteur spatial à un service externe (geofence-service de l'écosystème TraEnSys). Cette délégation est réalisée via le port sortant `ExternalGeofencePort`.

3.7.3 Modèles de Domaine

Listing 3.7 : Modèles GeofenceZone et GeofencePoint

```
1 public record GeofenceZone(  
2     UUID id, UUID fleetId, UUID managerId,  
3     String title, String description,  
4     String type, // POLYGON ou CIRCLE  
5     Double centerLatitude, Double centerLongitude,  
6     Double radius, // Pour les cercles (en mètres)  
7     Boolean isTemporalEnabled,  
8     LocalTime startTime, LocalTime endTime,  
9     String remoteId, // ID dans le service externe  
10    Boolean isConditionalEnabled,  
11    // ... autres champs de configuration  
12    Boolean isActive,  
13    List<GeofencePoint> vertices // Sommets du polygone  
14 ) {}  
15  
16 public record GeofencePoint(  
17     UUID id, Double latitude, Double longitude, Integer vertexOrder  
18 ) {}
```

3.7.4 Endpoints REST du Module Geofence

Méthode URL		Rôle requis	Description
POST	/api/v1/geofence/zones	MANAGER/ADMIN	Créer une zone
GET	/api/v1/geofence/circles	MANAGER	Zones circulaires
GET	/api/v1/geofence/polygons	MANAGER	Zones polygonales
GET	/api/v1/geofence/my-zones	MANAGER	Mes zones
GET	/api/v1/geofence/fleet/{fleetId}	MANAGER	Zones d'une flotte

Méthode	URL	Rôle requis	Description
GET	/api/v1/geofence/zones/{id}	Authentifié	Détails d'une zone
PUT	/api/v1/geofence/{type}/{id}	MANAGER/ADMIN	Modifier une zone
PATCH	/api/v1/geofence/{id}/assign-fleet/{fleetId}	Authentifié	Assigner à une flotte
DELETE	/api/v1/geofence/{type}/{id}	MANAGER/ADMIN	Supprimer une zone
GET	/api/v1/geofence/alerts	MANAGER	Mes alertes
GET	/api/v1/geofence/alerts/all	Authentifié	Toutes les alertes

TAB. 3.6 : Endpoints REST du module Geofence

3.8 Module de Notifications

Le module de notifications permet l'envoi d'alertes aux utilisateurs. Il est implémenté via le port sortant `SendNotificationPort`, avec deux modes configurables :

- **Mode HTTP** : Délégation au service de notification externe de l'écosystème TraEnSys via `HttpNotificationAdapter`
- **Mode local** : Stockage en base de données pour consultation ultérieure

Les templates de notification sont configurés dans `application.yml` avec des IDs numériques :

Listing 3.8 : Configuration des templates de notification

```
application:
  notification:
    templates:
      new-offer: 1
      driver-applied: 2
      driver-selected: 3
      ride-confirmed: 4
      ride-cancelled: 5
      admin-validation: 7
      maintenance-created: 8
      incident-reported: 9
      incident-critical: 10
```

3.9 Module d'Administration

3.9.1 Administrateur

Le module administrateur (`AdminService`, `AdminManagerController`) permet :

- La gestion des comptes gestionnaires de flotte (création, suspension, suppression)
- La supervision globale de la plateforme
- L'envoi de notifications système à tous les utilisateurs

3.9.2 Super Administrateur

Le super administrateur (`SuperAdminService`, `SuperAdminController`) dispose des droits les plus élevés :

- Gestion des administrateurs
- Accès aux statistiques globales via `AdminStatsService`
- Initialisation du système (compte créé automatiquement au démarrage via `SystemUserInitializer`)

3.9.3 Initialisation Automatique

La classe `SystemUserInitializer` crée automatiquement le compte super-administrateur au premier démarrage si celui-ci n'existe pas. Les credentials sont configurables via `application.yml` :

Listing 3.9 : Configuration du super-administrateur initial

```
application:
  bootstrap:
    admin:
      username: "superadmin123"
      email: "superadmin123@transens.com"
      password: "password123"
      phone: "+23761213141516"
      firstname: "System"
      lastname: "Administrator"
```

Chapitre 4

La Couche Infrastructure Existante

4.1 La Base de Données PostgreSQL

4.1.1 Organisation en Schémas

La base de données est organisée en deux schémas distincts :

- **Schéma public** : Tables partagées avec d'autres services de l'écosystème (notamment la table `users` gérée par le service Auth)
- **Schéma fleet** : Tables propres au service FleetMan

4.1.2 Migrations Liquibase

Les migrations sont gérées par Liquibase via un fichier maître `db.changelog-master.yaml` qui orchestre l'exécution séquentielle des changesets :

Fichier	Contenu
001-init-schemas.sql	Création des schémas <code>public</code> et <code>fleet</code>
002-fleet-tables.sql	Tables principales : véhicules, conducteurs, flottes, trajets, géorepérage
003-create-notifications-tables.sql	Tables de notifications et historique de notifications
004-update_geofence_tables.sql	Mise à jour des tables de géorepérage
005-move-users-to-fleetman.sql	Migraion des utilisateurs vers le schéma <code>fleet</code>
006-add-photo-url-to-users.sql	Ajout d'un champ <code>photo_url</code> aux utilisateurs
007-add-geofence-remote-id.sql	Ajout d'un ID distant pour les zones géographiques
008-add-vehicle-resources.sql	Tables référentiels véhicule
009-add-geofence-details.sql	Détails supplémentaires des zones géographiques
010-operations-tables.sql	[AJOUT] Tables du module Opérations Terrain

TAB. 4.1 : Historique des migrations Liquibase

4.1.3 Schéma de la Base de Données Principale

Le schéma `fleet` contient les tables suivantes avant nos ajouts :

Table	Description
<code>fleet.vehicle_types</code>	Référentiel des types de véhicules
<code>fleet.fleet_managers</code>	Profil des gestionnaires de flotte
<code>fleet.fleets</code>	Flottes de véhicules
<code>fleet.drivers</code>	Profil des conducteurs
<code>fleet.vehicles</code>	Véhicules avec leurs caractéristiques
<code>fleet.vehicle_illustrations</code>	Galerie images des véhicules
<code>fleet.operational_parameters</code>	Télé-métrie temps réel (1-1 avec vehicle)
<code>fleet.financial_parameters</code>	Paramètres financiers (1-1 avec vehicle)
<code>fleet.maintenance_parameters</code>	Paramètres de maintenance (1-1 avec vehicle)
<code>fleet.trips</code>	Trajets effectués
<code>fleet.geofence_zones</code>	Zones géographiques
<code>fleet.geofence_points</code>	Points GPS des zones
<code>fleet.geofence_point_zones</code>	Associations points-zones
<code>fleet.geofence_events</code>	Événements d'entrée/sortie de zones
<code>fleet.routes</code>	Routes des trajets

TAB. 4.2 : Tables du schéma `fleet` avant les ajouts

4.2 Les Adapters de Persistance R2DBC

4.2.1 Pattern Adapter de Persistance

Chaque adapter de persistance suit le même pattern :

Listing 4.1 : Pattern d'un adapter de persistance R2DBC

```

1 @Component
2 @RequiredArgsConstructor
3 public class VehiclePersistenceAdapter implements
    VehiclePersistencePort {
4
5     private final VehicleLocalR2dbcRepository vehicleRepo;
6     private final VehicleLocalMapper mapper;
7
8     @Override
9     public Mono<Vehicle> save(Vehicle vehicle) {
10         VehicleLocalEntity entity = mapper.toEntity(vehicle);

```



```
11         return vehicleRepo.save(entity)
12             .map(mapper::toDomain);
13     }
14
15     @Override
16     public Mono<Vehicle> findById(UUID id) {
17         return vehicleRepo.findById(id)
18             .map(mapper::toDomain)
19             .switchIfEmpty(Mono.error(
20                 VehicleException.notFound(id)));
21     }
22     // ...
23 }
```

4.2.2 Repositories Spring Data R2DBC

Les repositories étendent `ReactiveCrudRepository<Entity, UUID>` et définissent des requêtes personnalisées via l'annotation `@Query` :

Listing 4.2 : Exemple de repository R2DBC avec requêtes personnalisées

```
1 public interface VehicleLocalR2dbcRepository
2     extends ReactiveCrudRepository<VehicleLocalEntity, UUID> {
3
4     @Query("SELECT * FROM fleet.vehicles WHERE manager_id = :managerId"
5         )
6     Flux<VehicleLocalEntity> findByManagerId(UUID managerId);
7
8     @Query("SELECT * FROM fleet.vehicles WHERE fleet_id = :fleetId")
9     Flux<VehicleLocalEntity> findByFleetId(UUID fleetId);
10
11     @Query("SELECT * FROM fleet.vehicles WHERE license_plate = :plate")
12     Mono<VehicleLocalEntity> findByLicensePlate(String plate);
13 }
```

4.3 Les Adapters Externes (HTTP Clients)

4.3.1 Architecture des Clients HTTP

Le projet communique avec plusieurs services externes via des clients HTTP basés sur `WebClient` (non-bloquant) :

Client	Service Cible	Rôle
AuthApiClient	Service Auth	Authentification, gestion des comptes
VehicleApiClient	Service Véhicule	Enregistrement distant des véhicules
GeofenceApiClient	Service Geofence	Gestion des zones géographiques
GeofenceAuthClient	Service Auth	Authentification pour le geofence
NotificationApiClient	Service Notification	Envoi de notifications push

TAB. 4.3 : Clients HTTP externes du projet

4.3.2 Adapters Externes

Chaque client HTTP est encapsulé dans un adapter qui implémente le port sortant correspondant :

- RemoteAuthAdapter : Implémente AuthPort via AuthApiClient
- FakeAuthAdapter : Implémentation de test (mode local sans service Auth)
- VehicleApiAdapter : Implémente ExternalVehiclePort
- GeofenceApiAdapter : Implémente ExternalGeofencePort
- PaymentApiAdapter : Implémente ExternalPaymentPort
- FakeDistanceAdapter : Implémentation de test pour le calcul de distances

4.4 Les Adapters Kafka

4.4.1 Configuration Kafka

Kafka est configuré dans KafkaConfig avec deux topics principaux :

Listing 4.3 : Configuration Kafka dans application.yml

```
spring:
  kafka:
    bootstrap-servers: localhost:9092
    consumer:
      group-id: fleet-management-group
      auto-offset-reset: earliest
```

```
producer:
  key-serializer: StringSerializer
  value-serializer: JsonSerializer

application:
  kafka:
    topics:
      geofence-alerts: geofence-alerts-topic
      operation-events: operation-events-topic
```

4.4.2 Producteurs et Consommateurs

- **KafkaOperationEventAdapter** : Producteur — publie les événements de maintenance et d'incident sur le topic `operation-events-topic`
- **GeofenceAlertEventPublisherAdapter** : Producteur — publie les alertes de géorepérage sur `geofence-alerts-topic`
- **OperationEventConsumer** : Consommateur — traite les événements opérationnels reçus

4.5 Les Mappers MapStruct

MapStruct est utilisé pour la conversion entre les entités R2DBC (couche infrastructure) et les modèles de domaine (couche domaine). Quatre mappers sont définis :

- **VehicleLocalMapper** : `VehicleLocalEntity` ↔ `Vehicle`
- **DriverMapper** : `DriverEntity` ↔ `Driver`
- **FleetMapper** : `FleetEntity` ↔ `Fleet`
- **GeofenceMapper** : `GeofenceZoneEntity` ↔ `GeofenceZone`

4.6 Configuration du Monitoring

4.6.1 Spring Actuator

Les endpoints Actuator exposés sont : `health`, `info`, `prometheus`. L'endpoint `health` expose les détails complets et les probes Kubernetes (`liveness/readiness`).

4.6.2 Métriques Prometheus

Micrometer avec le registre Prometheus est configuré pour exporter les métriques de l'application. Ces métriques peuvent être collectées par un serveur Prometheus et visualisées dans Grafana.

4.6.3 Circuit Breaker Resilience4j

Resilience4j est intégré via Spring Cloud Circuit Breaker pour protéger les appels aux services externes. En cas de défaillance répétée d'un service externe, le circuit s'ouvre et les appels sont court-circuités, évitant la propagation des erreurs.

Chapitre 5

Configuration Système et Déploiement

5.1 Configuration de l'Application

5.1.1 Profils Spring

Le projet utilise deux profils Spring :

- **Profil `local`** : Configuration pour le développement local avec Docker Compose
- **Profil `prod`** : Configuration pour l'environnement de production (fichier `prod.application.yml`)

5.1.2 Configuration des Services Externes

Listing 5.1 : Configuration des services externes

```
application:
  external:
    vehicle-service-url: https://traefikdev.yowyob.com/vehicule
    geofence-service-url: https://traefikdev.yowyob.com/geofence-
      service
    payment-service-url: https://traefikdev.yowyob.com/payment
  auth:
    mode: remote
    url: https://traefikdev.yowyob.com/auth
  notification:
    mode: http
    url: https://traefikdev.yowyob.com/notification
```

5.2 Containerisation Docker

5.2.1 Dockerfile

Le projet est containerisé via un Dockerfile multi-stage optimisé pour la production. L'image finale est basée sur une JRE 21 allégée.

5.2.2 Docker Compose

Le fichier `docker-compose.yml` orchestre les services nécessaires au développement local :

- **PostgreSQL** : Base de données principale (port 5433)
- **Redis** : Cache télémétrie (port 6380)
- **Apache Kafka** : Messagerie asynchrone (port 9092)
- **Zookeeper** : Coordination Kafka

5.3 CI/CD

Un pipeline GitHub Actions est configuré dans `.github/workflows/ci-cd.yml` pour automatiser la compilation, les tests et le déploiement.

5.4 Documentation API

La documentation de l'API est générée automatiquement par SpringDoc OpenAPI 2.5.0 et accessible via :

- **Swagger UI** : `http://localhost:8080/swagger-ui.html`
- **OpenAPI JSON** : `http://localhost:8080/v3/api-docs`

Les tags Swagger sont organisés par module fonctionnel, numérotés de 01 à 14 pour faciliter la navigation.

Deuxième partie

Les Apports : Module Opérations Terrain

Chapitre 6

Analyse des Besoins et Conception du Module Opérations

6.1 Contexte et Motivation

Avant nos contributions, le backend FleetMan disposait d'une architecture solide pour la gestion des véhicules, des conducteurs, des trajets et du géorepérage. Cependant, un aspect crucial de la gestion de flotte était absent : la **traçabilité des opérations terrain**.

Dans la réalité opérationnelle d'une entreprise de transport, trois types d'événements surviennent quotidiennement et nécessitent un suivi rigoureux :

1. **Les maintenances** : Interventions techniques planifiées ou correctives sur les véhicules (vidange, remplacement de pièces, réparations)
2. **Les incidents** : Événements imprévus pouvant affecter la sécurité ou la disponibilité des véhicules (accidents, pannes, vols, infractions)
3. **Les recharges de carburant** : Pleins effectués par les conducteurs, représentant un poste de coût majeur à maîtriser

L'absence de ces fonctionnalités rendait impossible la construction de tableaux de bord opérationnels complets et la maîtrise des coûts d'exploitation.

6.2 Analyse des Besoins Fonctionnels

6.2.1 Besoins liés aux Mainténances

- Enregistrer une intervention technique avec son objet, son coût, son rapport et sa localisation GPS
- Consulter l'historique des maintenances par véhicule, par conducteur ou par période

- Mettre à jour automatiquement les paramètres de maintenance du véhicule après chaque intervention
- Notifier le gestionnaire de flotte lors de la déclaration d'une maintenance

6.2.2 Besoins liés aux Incidents

- Déclarer un incident avec son type, sa description, sa sévérité et sa localisation
- Gérer le cycle de vie complet d'un incident : REPORTED → UNDER_INVESTIGATION → RESOLVED → CLOSED
- Filtrer les incidents par type, sévérité, statut ou période
- Calculer les coûts totaux des incidents par véhicule ou par conducteur (KPIs)
- Déclencher des alertes prioritaires pour les incidents critiques (HIGH ou CRITICAL)
- Enregistrer les informations de témoins et les numéros officiels (PV police, déclaration assurance)

6.2.3 Besoins liés aux Recharges Carburant

- Enregistrer un plein avec la quantité, le prix, la station et la localisation GPS
- Calculer automatiquement le coût unitaire (prix par litre)
- Mettre à jour automatiquement le niveau de carburant du véhicule après chaque recharge
- Calculer la consommation totale et les coûts carburant par véhicule (KPIs)

6.3 Analyse des Besoins Non-Fonctionnels

- **Cohérence architecturale** : Les nouveaux modules doivent respecter scrupuleusement l'architecture hexagonale réactive existante
- **Réactivité** : Toutes les opérations doivent être non-bloquantes (Mono/-Flux)

- **Isolation** : Le module Opérations ne doit pas créer de couplage fort avec les autres modules
- **Sécurité** : Les données opérationnelles sont cloisonnées par gestionnaire (un manager ne voit que ses propres données)
- **Intégrité** : Les invariants métier doivent être validés dans le domaine, pas dans les services

6.4 Identification des Agrégats DDD

L'analyse des besoins a conduit à identifier trois agrégats distincts dans le bounded context *Opérations Terrain* :

Agrégat	Racine	Invariants Métier
Maintenance	Maintenance	Sujet obligatoire, véhicule obligatoire, coût ≥ 0
Incident	Incident	Type obligatoire, véhicule obligatoire, machine à états
FuelRecharge	FuelRecharge	Quantité > 0 , prix ≥ 0 , véhicule obligatoire

TAB. 6.1 : Agrégats DDD du module Opérations Terrain

6.5 Stratégie d'Intégration

Le module Opérations Terrain a été développé dans le projet `Fleetman Backend` (architecture Spring MVC classique) puis intégré dans `fleet-management-back` (architecture hexagonale réactive). Cette intégration a nécessité plusieurs adaptations :

Point d'adaptation	Transformation appliquée
IDs de type Long	Remplacés par UUID (cohérence avec le projet principal)
Paradigme synchrone (<code>List</code> , <code>Optional</code>)	Adapté en réactif (<code>Flux</code> , <code>Mono</code>)
Package	Migré vers <code>com.yowjob.fleet</code>
Annotations JPA dans le domaine	Supprimées (domaine pur)

Point d'adaptation	Transformation appliquée
Migrations SQL absentes	Créées en changesets Liquibase

TAB. 6.2 : Adaptations réalisées lors de l'intégration

Chapitre 7

Implémentation — Couche Domaine

7.1 Le Value Object Coordinates

7.1.1 Conception

Le premier apport au domaine est le value object `Coordinates`, un record Java immuable représentant une position géographique. Ce concept était absent du domaine existant, qui stockait les coordonnées de façon dispersée dans différentes entités.

Listing 7.1 : Value Object Coordinates — domaine pur

```
1 public record Coordinates(double longitude, double latitude) {
2
3     public Coordinates {
4         if (longitude < -180 || longitude > 180) {
5             throw new IllegalArgumentException(
6                 "Longitude invalide : " + longitude +
7                 ". Doit etre comprise entre -180 et 180.");
8         }
9         if (latitude < -90 || latitude > 90) {
10            throw new IllegalArgumentException(
11                "Latitude invalide : " + latitude +
12                ". Doit etre comprise entre -90 et 90.");
13        }
14    }
15
16    @Override
17    public String toString() {
18        return "POINT(" + longitude + " " + latitude + ")";
19    }
20 }
```

7.1.2 Caractéristiques Techniques

- **Record Java immuable** : Garantit l'immuabilité des coordonnées une fois créées

- **Validation dans le bloc compact** : Les bornes géographiques sont vérifiées à la construction, jamais dans un service externe
- **Format WKT** : La méthode `toString()` retourne le format *Well-Known Text* compatible PostGIS (`POINT(lon lat)`)
- **Domaine pur** : Aucune dépendance externe (les librairies `ugeojson` du dossier `libs` ne sont pas utilisées dans le domaine)

7.2 L'Entité Maintenance

7.2.1 Conception

L'entité `Maintenance` représente une intervention technique sur un véhicule. Contrairement aux modèles de domaine existants (records immuables), `Maintenance` est une classe Java avec état mutable, car elle dispose de méthodes métier qui modifient son état.

7.2.2 Invariants Métier

Les invariants sont validés dans le constructeur :

Listing 7.2 : Invariants métier de Maintenance

```
1 public Maintenance(UUID id, String subject, BigDecimal cost,
2                     LocalDateTime dateTime, String report,
3                     Coordinates location, String locationName,
4                     UUID vehicleId, String vehicleRegistrationNumber,
5                     UUID driverId, String driverFullName) {
6
7     if (subject == null || subject.isBlank()) {
8         throw new IllegalArgumentException(
9             "L'objet de la maintenance est obligatoire.");
10    }
11    if (vehicleId == null) {
12        throw new IllegalArgumentException(
13            "Le vehicule est obligatoire pour une maintenance.");
14    }
15    // ... initialisation des champs
16 }
```

7.2.3 Méthodes Métier

Méthode	Comportement
<code>addReport(String)</code>	Ajoute ou met à jour le rapport d'intervention
<code>updateCost(BigDecimal)</code>	Met à jour le coût (validation : coût ≥ 0)
<code>updateLocation(Coordinates, String)</code>	Met à jour la localisation GPS et le nom du lieu

TAB. 7.1 : Méthodes métier de l'entité Maintenance

7.2.4 Données Dénormalisées

L'entité stocke des données dénormalisées pour l'affichage (`vehicleRegistrationNumber`, `driverFullName`), évitant des jointures supplémentaires. Ce pattern est déjà utilisé dans le projet principal.

7.3 L'Entité Incident — Machine à États

7.3.1 Conception

L'entité `Incident` est la plus complexe du module. Elle implémente une **machine à états** complète avec des méthodes métier expressives.

7.3.2 Enums du Domaine

Listing 7.3 : Enums imbriqués de l'entité Incident

```

1 public class Incident {
2
3     public enum Type {
4         ACCIDENT, BREAKDOWN, THEFT,
5         VANDALISM, TRAFFIC_VIOLATION, OTHER
6     }
7
8     public enum Severity {
9         LOW, MEDIUM, HIGH, CRITICAL
10    }
11
12    public enum Status {
13        REPORTED, UNDER_INVESTIGATION, RESOLVED, CLOSED
14    }
15 }
```

7.3.3 Machine à États

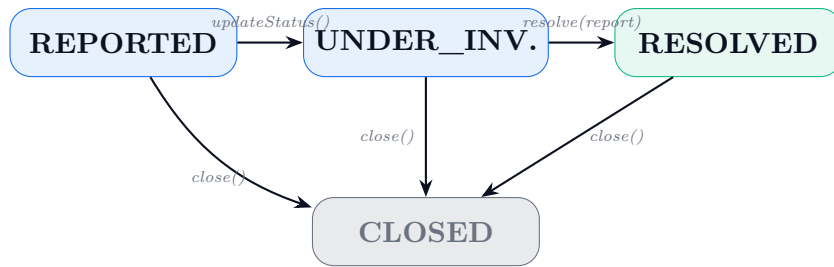


FIG. 7.1 : Machine à états de l'entité Incident

7.3.4 Méthodes Métier

Listing 7.4 : Méthodes métier de la machine à états Incident

```

1 public void resolve(String report) {
2     if (this.status == Status.CLOSED) {
3         throw new IllegalStateException(
4             "Un incident cloture ne peut pas etre resolu.");
5     }
6     this.status = Status.RESOLVED;
7     this.report = report;
8     this.resolvedAt = LocalDateTime.now();
9 }
10
11 public void close() {
12     this.status = Status.CLOSED;
13     if (this.resolvedAt == null) {
14         this.resolvedAt = LocalDateTime.now();
15     }
16 }
17
18 public boolean isCritical() {
19     return this.severity == Severity.CRITICAL
20         || this.severity == Severity.HIGH;
21 }
22
23 public boolean isOpen() {
24     return this.status == Status.REPORTED
25         || this.status == Status.UNDER_INVESTIGATION;
26 }

```

7.4 L'Entité FuelRecharge

7.4.1 Conception

L'entité `FuelRecharge` représente un plein de carburant. Ses champs `quantity` et `price` sont `final` (invariants métier), ce qui garantit l'intégrité des données financières.

7.4.2 Méthode Métier `unitCost()`

Listing 7.5 : Calcul du coût unitaire

```

1 public BigDecimal unitCost() {
2     if (quantity.compareTo(BigDecimal.ZERO) == 0) {
3         return BigDecimal.ZERO;
4     }
5     return price.divide(quantity, 2, RoundingMode.HALF_UP);
6 }

```

Cette méthode calcule le coût par litre avec une précision de 2 décimales et une protection contre la division par zéro.

7.4.3 Enum `StationName`

L'enum `StationName` liste les stations-service disponibles dans le contexte camerounais : `TOTAL`, `SHELL`, `OILIBYA`, `CAMRAIL`, `OTHER`.

7.5 L'Exception Métier `OperationException`

7.5.1 Conception

`OperationException` hérite de `DomainException` (pattern identique aux exceptions existantes `VehicleException`, `TripException`, etc.) et définit des codes d'erreur typés :

Code	HTTP	Description
OPR_001	404	Maintenance introuvable
OPR_002	400	Sujet de maintenance obligatoire
OPR_003	400	Coût négatif interdit
OPR_004	404	Incident introuvable
OPR_005	400	Type d'incident obligatoire
OPR_006	422	Incident déjà clôturé
OPR_007	422	Transition de statut invalide
OPR_008	404	Recharge de carburant introuvable

Code	HTTP	Description
OPR_009	400	Quantité de carburant invalide
OPR_010	400	Prix de recharge invalide
OPR_011	404	Véhicule introuvable pour l'opération
OPR_012	404	Chauffeur introuvable pour l'opération

TAB. 7.2 : Codes d'erreur de OperationException

Chapitre 8

Implémentation — Ports et Services Applicatifs

8.1 Les Ports Entrants (Use Cases)

8.1.1 ManageMaintenanceUseCase

L'interface `ManageMaintenanceUseCase` définit 9 méthodes réactives couvrant l'ensemble du cycle de vie des maintenances :

Listing 8.1 : Interface `ManageMaintenanceUseCase`

```
1 public interface ManageMaintenanceUseCase {
2
3     Mono<Maintenance> createMaintenance(CreateMaintenanceCommand cmd);
4     Mono<Maintenance> getById(UUID id);
5     Flux<Maintenance> getAllByManager(UUID managerId);
6     Flux<Maintenance> getByVehicleId(UUID vehicleId);
7     Flux<Maintenance> getByDriverId(UUID driverId);
8     Flux<Maintenance> getByDateRange(LocalDateTime start,
9                                     LocalDateTime end,
10                                    UUID managerId);
11     Mono<Long> countByDriverId(UUID driverId);
12     Mono<Maintenance> update(UpdateMaintenanceCommand cmd);
13     Mono<Void> delete(UUID id);
14
15     record CreateMaintenanceCommand(
16         String subject, BigDecimal cost, String report,
17         Double longitude, Double latitude, String locationName,
18         UUID vehicleId, UUID driverId
19     ) {}
20
21     record UpdateMaintenanceCommand(
22         UUID maintenanceId, String subject, BigDecimal cost,
23         String report, Double longitude, Double latitude,
24         String locationName
25     ) {}
```

```
26 }
```

8.1.2 ManageIncidentUseCase

L'interface la plus riche du module avec 16 méthodes réactives, incluant des KPIs financiers :

Listing 8.2 : Méthodes KPI de ManageIncidentUseCase

```
1 Flux<Incident> getByType(Incident.Type type, UUID managerId);
2 Flux<Incident> getBySeverity(Incident.Severity severity, UUID managerId
  );
3 Flux<Incident> getByStatus(Incident.Status status, UUID managerId);
4 Flux<Incident> getOpenIncidents(UUID managerId);
5 Mono<Incident> updateStatus(UUID id, Incident.Status status);
6 Mono<Long> countByVehicleId(UUID vehicleId);
7 Mono<BigDecimal> getTotalCostByVehicleId(UUID vehicleId);
8 Mono<BigDecimal> getTotalCostByDriverId(UUID driverId);
```

8.1.3 ManageFuelRechargeUseCase

L'interface ManageFuelRechargeUseCase définit 10 méthodes réactives avec des agrégats de consommation :

Listing 8.3 : Méthodes KPI de ManageFuelRechargeUseCase

```
1 Mono<BigDecimal> getTotalQuantityByVehicleId(UUID vehicleId);
2 Mono<BigDecimal> getTotalCostByVehicleId(UUID vehicleId);
```

8.2 Les Ports Sortants

8.2.1 OperationEventPort — Port Événementiel

Ce port découplé permet au domaine de publier des événements sans dépendre de Spring ni de Kafka :

Listing 8.4 : Port événementiel OperationEventPort

```
1 public interface OperationEventPort {
2
3     Mono<Void> publishMaintenanceCreated(MaintenanceCreatedEvent event)
4         ;
5     Mono<Void> publishIncidentReported(IncidentReportedEvent event);
6
7     record MaintenanceCreatedEvent(
8         UUID maintenanceId, String subject,
```

```

8      UUID vehicleId, String vehicleRegistration,
9      UUID driverId, UUID fleetManagerId
10    ) {}
11
12    record IncidentReportedEvent(
13      UUID incidentId, String incidentType, String severity,
14      UUID vehicleId, String vehicleRegistration,
15      UUID driverId, UUID fleetManagerId, boolean isCritical
16    ) {}
17  }

```

Le champ `fleetManagerId` dans chaque événement permet de router les notifications vers le bon gestionnaire.

8.3 Les Services Applicatifs

8.3.1 MaintenanceService

Le service `MaintenanceService` implémente `ManageMaintenanceUseCase` et orchestre la logique métier de création d'une maintenance :

Listing 8.5 : Logique de création d'une maintenance

```

1  @Override
2  public Mono<Maintenance> createMaintenance(CreateMaintenanceCommand cmd) {
3      return vehiclePersistencePort.findById(cmd.vehicleId())
4          .switchIfEmpty(Mono.error(
5              OperationException.vehicleNotFound(cmd.vehicleId())))
6          .flatMap(vehicle -> {
7              // Verification optionnelle du chauffeur
8              Mono<Driver> driverMono = cmd.driverId() != null
9                  ? driverPersistencePort.findById(cmd.driverId())
10                 .switchIfEmpty(Mono.error(
11                     OperationException.driverNotFound(cmd.driverId())
12                     ))
13                 : Mono.empty();
14
15              return driverMono.defaultIfEmpty(null)
16                 .flatMap(driver -> {
17                     // Construction avec invariants
18                     Coordinates coords = buildCoordinates(
19                         cmd.longitude(), cmd.latitude());
20                     Maintenance maintenance = new Maintenance(
21                         null, cmd.subject(), cmd.cost(),
22                         LocalDateTime.now(), cmd.report(),

```

```

22         coords, cmd.locationName(),
23         vehicle.id(), vehicle.licensePlate(),
24         driver != null ? driver.userId() : null,
25         driver != null ? /* nom */ null : null
26     );
27     return maintenancePersistencePort.save(maintenance)
28         .flatMap(saved -> syncMaintenanceParams(saved)
29             .thenReturn(saved))
30         .doOnSuccess(saved ->
31             publishMaintenanceEvent(saved, vehicle)
32                 .subscribe());
33     });
34 });
35 }

```

8.3.2 Synchronisation Inter-Modules

Après chaque création de maintenance, le service met à jour automatiquement les paramètres de maintenance du véhicule :

Listing 8.6 : Synchronisation des paramètres de maintenance

```

1 private Mono<Void> syncMaintenanceParams(Maintenance saved) {
2     return maintenanceParameterRepo
3         .findByVehicleId(saved.getVehicleId())
4         .flatMap(params -> {
5             params.setLastMaintenanceAt(
6                 saved.getDateTime().toLocalDate());
7             params.setMaintenanceStatus("UP_TO_DATE");
8             return maintenanceParameterRepo.save(params);
9         })
10        .then();
11 }

```

8.3.3 IncidentService — Gestion du Cycle de Vie

Le service IncidentService délègue les transitions d'état à la machine à états de l'entité :

Listing 8.7 : Délégation à la machine à états

```

1 @Override
2 public Mono<Incident> updateStatus(UUID id, Incident.Status newStatus)
3 {
4     return incidentPersistencePort.findById(id)
5         .switchIfEmpty(Mono.error(
6             OperationException.incidentNotFound(id)))

```

```

6      .flatMap(incident -> {
7          if (incident.getStatus() == Incident.Status.CLOSED) {
8              return Mono.error(
9                  OperationException.incidentAlreadyClosed(id));
10         }
11         switch (newStatus) {
12             case RESOLVED -> incident.resolve(null);
13             case CLOSED    -> incident.close();
14             default        -> incident.updateStatus(newStatus);
15         }
16         return incidentPersistencePort.save(incident);
17     });
18 }

```

Chapitre 9

Implémentation — Couche Infrastructure

9.1 Migration Liquibase — Changeset 010

Le changeset 010-operations-tables.sql crée les trois nouvelles tables dans le schéma fleet :

9.1.1 Table fleet.maintenances

Listing 9.1 : Création de la table fleet.maintenances

```
1 CREATE TABLE IF NOT EXISTS fleet.maintenances (
2     id                UUID PRIMARY KEY,
3     subject           VARCHAR(255) NOT NULL,
4     cost              NUMERIC(12, 2),
5     date_time         TIMESTAMP NOT NULL DEFAULT now(),
6     report            TEXT,
7     longitude         NUMERIC(10, 7),
8     latitude          NUMERIC(10, 7),
9     location_name     VARCHAR(255),
10    vehicle_id        UUID NOT NULL
11                        REFERENCES fleet.vehicles(id) ON DELETE
12                        CASCADE,
13    vehicle_registration VARCHAR(50),
14    driver_id         UUID
15                        REFERENCES fleet.drivers(user_id) ON DELETE
16                        SET NULL,
17    driver_full_name   VARCHAR(255)
18 );
19
20 CREATE INDEX IF NOT EXISTS idx_maintenances_vehicle_id
21     ON fleet.maintenances(vehicle_id);
22 CREATE INDEX IF NOT EXISTS idx_maintenances_driver_id
23     ON fleet.maintenances(driver_id);
24 CREATE INDEX IF NOT EXISTS idx_maintenances_date_time
```

```
23 ON fleet.maintenances(date_time DESC);
```

9.1.2 Table fleet.incidents

Listing 9.2 : Création de la table fleet.incidents

```
1 CREATE TABLE IF NOT EXISTS fleet.incidents (
2     id                                UUID PRIMARY KEY,
3     type                             VARCHAR(50) NOT NULL
4     CHECK (type IN ('ACCIDENT', 'BREAKDOWN', 'THEFT',
5                     'VANDALISM', 'TRAFFIC_VIOLATION', 'OTHER')),
6     description                      TEXT,
7     severity                         VARCHAR(50) DEFAULT 'MEDIUM'
8     CHECK (severity IN ('LOW', 'MEDIUM', 'HIGH', 'CRITICAL')),
9     incident_date_time               TIMESTAMP NOT NULL DEFAULT now(),
10    longitude                         NUMERIC(10, 7),
11    latitude                         NUMERIC(10, 7),
12    cost                             NUMERIC(12, 2),
13    status                           VARCHAR(50) DEFAULT 'REPORTED'
14    CHECK (status IN ('REPORTED', 'UNDER_INVESTIGATION',
15                     'RESOLVED', 'CLOSED')),
16    report                           TEXT,
17    witness_name                     VARCHAR(255),
18    witness_contact                  VARCHAR(255),
19    police_report_number              VARCHAR(100),
20    insurance_claim_number            VARCHAR(100),
21    reported_by                      VARCHAR(255),
22    resolved_at                      TIMESTAMP,
23    vehicle_id                       UUID NOT NULL
24    REFERENCES fleet.vehicles(id) ON DELETE CASCADE,
25    vehicle_registration              VARCHAR(50),
26    driver_id                        UUID
27    REFERENCES fleet.drivers(user_id) ON DELETE SET NULL,
28    driver_full_name                  VARCHAR(255)
29 );
```

9.1.3 Table fleet.fuel_recharges

Listing 9.3 : Création de la table fleet.fuel_recharges

```
1 CREATE TABLE IF NOT EXISTS fleet.fuel_recharges (
2     id                                UUID PRIMARY KEY,
3     quantity                         NUMERIC(10, 2) NOT NULL,
4     price                           NUMERIC(12, 2) NOT NULL,
5     recharge_date_time               TIMESTAMP NOT NULL DEFAULT now(),
6     longitude                         NUMERIC(10, 7),
7     latitude                         NUMERIC(10, 7),
```



```

8      station_name          VARCHAR(50)
9      CHECK (station_name IN ('TOTAL', 'SHELL', 'OILIBYA',
10                             'CAMRAIL', 'OTHER')),
11      vehicle_id            UUID NOT NULL
12      REFERENCES fleet.vehicles(id) ON DELETE CASCADE,
13      vehicle_registration  VARCHAR(50),
14      driver_id             UUID
15      REFERENCES fleet.drivers(user_id) ON DELETE SET NULL,
16      driver_full_name      VARCHAR(255)
17 );

```

9.1.4 Décisions de Conception des Tables

- **Enums en VARCHAR** : Les enums sont stockés en VARCHAR plutôt qu'en types ENUM PostgreSQL natifs, pour éviter les problèmes de conversion avec R2DBC et simplifier les migrations futures
- **Coordonnées en deux colonnes** : longitude et latitude sont stockées séparément (pas de type PostGIS POINT) pour éviter la dépendance à l'extension PostGIS dans les entités R2DBC
- **Clés étrangères avec comportements** : ON DELETE CASCADE pour les véhicules (si un véhicule est supprimé, ses opérations le sont aussi), ON DELETE SET NULL pour les conducteurs (si un conducteur est supprimé, l'opération est conservée)
- **Index stratégiques** : Index sur vehicle_id, driver_id et les colonnes de date pour optimiser les requêtes de filtrage les plus fréquentes

9.2 Les Entités R2DBC

9.2.1 Pattern des Entités

Les entités R2DBC implémentent `Persistable<UUID>` avec un flag `isNew` pour distinguer les insertions des mises à jour (pattern identique aux entités existantes `TripEntity`, `MaintenanceParameterEntity`) :

Listing 9.4 : Pattern d'entité R2DBC avec `Persistable`

```

1 @Table("fleet.maintenances")
2 @Data
3 @NoArgsConstructor
4 @AllArgsConstructor
5 public class MaintenanceEntity implements Persistable<UUID> {

```

```

6
7     @Id
8     private UUID id;
9     private String subject;
10    private BigDecimal cost;
11    private LocalDateTime dateTime;
12    private String report;
13    private BigDecimal longitude;
14    private BigDecimal latitude;
15    private String locationName;
16    private UUID vehicleId;
17    private String vehicleRegistration;
18    private UUID driverId;
19    private String driverFullName;
20
21    @Transient
22    private boolean isNew;
23
24    @Override
25    public boolean isNew() { return isNew; }
26 }

```

9.3 Les Repositories R2DBC

9.3.1 IncidentR2dbcRepository — Le Plus Riche

Le repository des incidents est le plus complet avec 12 méthodes dont 8 requêtes `@Query` avec JOIN sur `fleet.vehicles` :

Listing 9.5 : Requête avancées du repository Incident

```

1 public interface IncidentR2dbcRepository
2     extends ReactiveCrudRepository<IncidentEntity, UUID> {
3
4     @Query("SELECT i.* FROM fleet.incidents i " +
5           "JOIN fleet.vehicles v ON i.vehicle_id = v.id " +
6           "WHERE v.manager_id = :managerId " +
7           "ORDER BY i.incident_date_time DESC")
8     Flux<IncidentEntity> findAllByManagerId(UUID managerId);
9
10    @Query("SELECT i.* FROM fleet.incidents i " +
11          "JOIN fleet.vehicles v ON i.vehicle_id = v.id " +
12          "WHERE i.status IN ('REPORTED','UNDER_INVESTIGATION') " +
13          "AND v.manager_id = :managerId")
14    Flux<IncidentEntity> findOpenIncidentsByManagerId(UUID managerId);
15

```

```
16     @Query("SELECT COALESCE(SUM(cost), 0) FROM fleet.incidents " +
17           "WHERE vehicle_id = :vehicleId")
18     Mono<BigDecimal> getTotalCostByVehicleId(UUID vehicleId);
19
20     @Query("SELECT COALESCE(SUM(cost), 0) FROM fleet.incidents " +
21           "WHERE driver_id = :driverId")
22     Mono<BigDecimal> getTotalCostByDriverId(UUID driverId);
23 }
```

Note technique : L'utilisation de `COALESCE(SUM(...), 0)` évite les valeurs `null` dans les `Mono<BigDecimal>` des KPIs, cohérent avec le `.defaultIfEmpty(ZERO)` des services.

9.4 Les Adapters de Persistance

9.4.1 Conversion Entity ↔ Domain

Chaque adapter assure la conversion bidirectionnelle entre l'entité R2DBC et le modèle de domaine. La conversion la plus complexe est celle de `IncidentPersistenceAdapter` qui doit gérer les enums stockés en String :

Listing 9.6 : Conversion Entity vers Domain pour Incident

```
1 private Incident toDomain(IncidentEntity entity) {
2     Coordinates coords = null;
3     if (entity.getLongitude() != null && entity.getLatitude() != null)
4     {
5         coords = new Coordinates(
6             entity.getLongitude().doubleValue(),
7             entity.getLatitude().doubleValue()
8         );
9     }
10    Incident incident = new Incident(
11        entity.getId(),
12        Incident.Type.valueOf(entity.getType()),
13        entity.getDescription(),
14        entity.getSeverity() != null
15            ? Incident.Severity.valueOf(entity.getSeverity())
16            : Incident.Severity.MEDIUM,
17        entity.getIncidentDateTime(),
18        coords,
19        entity.getCost(),
20        entity.getReportedBy(),
21        entity.getVehicleId(),
22        entity.getVehicleRegistration(),
```

```
23     entity.getDriverId(),
24     entity.getDriverFullName()
25 );
26
27 // Restauration de l'etat complet
28 if (entity.getReport() != null)
29     incident.setReport(entity.getReport());
30 if (entity.getPoliceReportNumber() != null)
31     incident.setPoliceReportNumber(entity.getPoliceReportNumber());
32 if (entity.getWitnessName() != null)
33     incident.addWitness(entity.getWitnessName(),
34                         entity.getWitnessContact());
35
36 return incident;
37 }
```

9.5 L'Adapter Kafka — KafkaOperationEventAdapter

L'adapter Kafka implémente `OperationEventPort` via `ReactiveKafkaProducerTemplate` :

Listing 9.7 : Publication d'événements via Kafka réactif

```
1  @Component
2  @RequiredArgsConstructor
3  @Slf4j
4  public class KafkaOperationEventAdapter implements OperationEventPort {
5
6      private final ReactiveKafkaProducerTemplate<String, Object>
7          kafkaTemplate;
8
9      @Value("${application.kafka.topics.operation-events:operation-
10 events-topic}")
11     private String operationEventsTopic;
12
13     @Override
14     public Mono<Void> publishIncidentReported(IncidentReportedEvent
15         event) {
16         if (event.isCritical()) {
17             log.warn("INCIDENT CRITIQUE signale : vehicule={}, type={}"
18                 ,
19                 event.vehicleId(), event.incidentType());
20         }
21         return kafkaTemplate
22             .send(operationEventsTopic, event.incidentId().toString(),
23                 event)
24             .doOnSuccess(result -> log.debug(
```


Chapitre 10

Implémentation — API REST et Résultats

10.1 Les DTOs de Requête et Réponse

10.1.1 DTOs de Requête

Les DTOs de requête sont des records Java avec annotations de validation Jakarta :

Listing 10.1 : DTO de requête MaintenanceRequest

```
1 public record MaintenanceRequest(  
2     @NotBlank(message = "L'objet est obligatoire")  
3     @Schema(description = "Objet de l'intervention",  
4         example = "Vidange + filtre huile")  
5     String subject,  
6  
7     @PositiveOrZero(message = "Le cout ne peut pas etre negatif")  
8     @Schema(description = "Cout en FCFA", example = "45000.00")  
9     BigDecimal cost,  
10  
11     @Schema(description = "Rapport detaille de l'intervention")  
12     String report,  
13  
14     @Schema(description = "Longitude GPS", example = "3.8480")  
15     Double longitude,  
16  
17     @Schema(description = "Latitude GPS", example = "11.5021")  
18     Double latitude,  
19  
20     @Schema(description = "Nom du lieu", example = "Garage Central  
21         Yaounde")  
22     String locationName,  
23  
24     @NotNull(message = "Le vehicule est obligatoire")  
25     @Schema(description = "ID du vehicule")  
    UUID vehicleId,
```

```
26
27     @Schema(description = "ID du chauffeur (optionnel)")
28     UUID driverId
29 ) {}
```

10.1.2 DTOs de Réponse

Les DTOs de réponse incluent des méthodes de fabrique statiques `from(Domain)` :

Listing 10.2 : DTO de réponse MaintenanceResponse avec fabrique

```
1 public record MaintenanceResponse(
2     UUID id, String subject, BigDecimal cost,
3     LocalDateTime dateTime, String report,
4     LocationDto location,
5     UUID vehicleId, String vehicleRegistration,
6     UUID driverId, String driverFullName
7 ) {
8     public record LocationDto(
9         Double longitude, Double latitude, String locationName
10    ) {}
11
12    public static MaintenanceResponse from(Maintenance m) {
13        LocationDto loc = null;
14        if (m.getLocation() != null) {
15            loc = new LocationDto(
16                m.getLocation().longitude(),
17                m.getLocation().latitude(),
18                m.getLocationName()
19            );
20        }
21        return new MaintenanceResponse(
22            m.getId(), m.getSubject(), m.getCost(),
23            m.getDateTime(), m.getReport(), loc,
24            m.getVehicleId(), m.getVehicleRegistrationNumber(),
25            m.getDriverId(), m.getDriverFullName()
26        );
27    }
28 }
```

10.2 Les Controllers WebFlux

10.2.1 MaintenanceController

Listing 10.3 : Controller WebFlux MaintenanceController

```
1 @RestController
2 @RequestMapping("/api/v1/operations/maintenances")
3 @RequiredArgsConstructor
4 @Tag(name = OpenApiConfig.TAG_OPS_MAINTENANCE)
5 @SecurityRequirement(name = "bearerAuth")
6 public class MaintenanceController {
7
8     private final ManageMaintenanceUseCase maintenanceUseCase;
9
10    @PostMapping
11    @ResponseStatus(HttpStatus.CREATED)
12    @PreAuthorize("hasAnyRole('FLEET_MANAGER', 'FLEET_DRIVER')")
13    @Operation(summary = "Déclarer une maintenance",
14              description = "Notifie automatiquement le Fleet Manager.
15              ")
16    public Mono<MaintenanceResponse> create(
17        @Valid @RequestBody MaintenanceRequest request) {
18        ManageMaintenanceUseCase.CreateMaintenanceCommand cmd =
19            new ManageMaintenanceUseCase.CreateMaintenanceCommand(
20                request.subject(), request.cost(), request.report(),
21                request.longitude(), request.latitude(),
22                request.locationName(), request.vehicleId(),
23                request.driverId()
24            );
25        return maintenanceUseCase.createMaintenance(cmd)
26            .map(MaintenanceResponse::from);
27    }
28    // ... autres endpoints
29 }
```

10.2.2 Récapitulatif des Endpoints du Module Opérations

Endpoints Maintenance

Méthode	URL	Rôle requis	HTTP
POST	/api/v1/operations/maintenances	MANAGER/DRIVER	201
GET	/api/v1/operations/maintenances	MANAGER	200
GET	/api/v1/operations/maintenances/{id}	MANAGER/DRIVER	200
GET	/api/v1/operations/maintenances/{vehicleId}	MANAGER	200
GET	/api/v1/operations/maintenances/{driverId}	MANAGER	200
GET	/api/v1/operations/maintenances/{range}	MANAGER	200
PUT	/api/v1/operations/maintenances/{id}	MANAGER	200
DELETE	/api/v1/operations/maintenances/{id}	MANAGER	204

Méthode	URL	Rôle requis	HTTP
---------	-----	-------------	------

TAB. 10.1 : Endpoints REST du module Maintenance

Endpoints Incident

Méthode	URL	Rôle requis	HTTP
POST	/api/v1/operations/incidents	MANAGER/DRIVER	201
GET	/api/v1/operations/incidents	MANAGER	200
GET	/api/v1/operations/incidents/{id}	MANAGER/DRIVER	200
GET	/api/v1/operations/incidents/vehicle/{vehicleId}	MANAGER	200
GET	/api/v1/operations/incidents/driver/{driverId}	MANAGER	200
GET	/api/v1/operations/incidents/open	MANAGER	200
GET	/api/v1/operations/incidents/filter	MANAGER	200
GET	/api/v1/operations/incidents/range	MANAGER	200
GET	/api/v1/operations/incidents/vehicle/{vehicleId}/cost	MANAGER	200
PUT	/api/v1/operations/incidents/{id}	MANAGER	200
PATCH	/api/v1/operations/incidents/{id}	MANAGER	200
DELETE	/api/v1/operations/incidents/{id}	MANAGER	204

TAB. 10.2 : Endpoints REST du module Incident

Endpoints Recharge Carburant

Méthode	URL	Rôle requis	HTTP
POST	/api/v1/operations/fuel-recharge	DRIVER	201
GET	/api/v1/operations/fuel-recharges	MANAGER	200
GET	/api/v1/operations/fuel-recharges/{id}	MANAGER/DRIVER	200
GET	/api/v1/operations/fuel-recharges/vehicle/{vehicleId}	MANAGER	200
GET	/api/v1/operations/fuel-recharges/driver/{driverId}	MANAGER	200
GET	/api/v1/operations/fuel-recharges/range	MANAGER	200
PUT	/api/v1/operations/fuel-recharges/{id}	MANAGER	200
DELETE	/api/v1/operations/fuel-recharges/{id}	MANAGER	204

TAB. 10.3 : Endpoints REST du module Recharge Carburant

10.3 Bilan des Fichiers Créés

10.3.1 Récapitulatif Global

Le tableau suivant liste l'ensemble des fichiers créés dans le cadre de ce projet :

Fichier	Couche	Rôle
domain/model/Coordinates.java	Domaine	Value Object GPS
domain/model/Maintenance.java	Domaine	Entité Maintenance
domain/model/Incident.java	Domaine	Entité Incident
domain/model/FuelRecharge.java	Domaine	Entité Recharge
domain/exception/OperationException.java	Domaine	Exception métier
domain/ports/in/ManageMaintenanceUseCase.java	Domaine	Port entrant
domain/ports/in/ManageIncidentUseCase.java	Domaine	Port entrant
domain/ports/in/ManageFuelRechargeUseCase.java	Domaine	Port entrant
domain/ports/out/MaintenancePersistencePort.java	Domaine	Port sortant
domain/ports/out/IncidentPersistencePort.java	Domaine	Port sortant
domain/ports/out/FuelRechargePersistencePort.java	Domaine	Port sortant
domain/ports/out/OperationEventPort.java	Domaine	Port événementiel
application/service/MaintenanceServiceApplication.java	Application	Service métier
application/service/IncidentServiceApplication.java	Application	Service métier
application/service/FuelRechargeServiceApplication.java	Application	Service métier
persistence/entity/MaintenanceEntityInfra.java	Infrastructure	Entité R2DBC
persistence/entity/IncidentEntityInfra.java	Infrastructure	Entité R2DBC
persistence/entity/FuelRechargeEntityInfra.java	Infrastructure	Entité R2DBC
persistence/repository/MaintenanceEntityRepository.java	Infrastructure	Repository
persistence/repository/IncidentEntityRepository.java	Infrastructure	Repository
persistence/repository/FuelRechargeEntityRepository.java	Infrastructure	Repository
persistence/MaintenancePersistenceAdapterInfra.java	Infrastructure	Adapter R2DBC
persistence/IncidentPersistenceAdapterInfra.java	Infrastructure	Adapter R2DBC
persistence/FuelRechargePersistenceAdapterInfra.java	Infrastructure	Adapter R2DBC
messaging/KafkaOperationEventAdapterInfra.java	Infrastructure	Adapter Kafka
rest/dto/MaintenanceRequest.java	Infrastructure	DTO requête
rest/dto/MaintenanceResponse.java	Infrastructure	DTO réponse
rest/dto/IncidentRequest.java	Infrastructure	DTO requête
rest/dto/IncidentUpdateRequest.java	Infrastructure	DTO mise à jour
rest/dto/IncidentStatusRequest.java	Infrastructure	DTO statut
rest/dto/IncidentResponse.java	Infrastructure	DTO réponse
rest/dto/FuelRechargeRequest.java	Infrastructure	DTO requête

Fichier	Couche	Rôle
rest/dto/FuelRechargeResponse.java	Infrastructure	DTO réponse
rest/MaintenanceController.java	Infrastructure	Controller REST
rest/IncidentController.java	Infrastructure	Controller REST
rest/FuelRechargeController.java	Infrastructure	Controller REST
db/changelog/changes/010-operations- tables.sql	Base de données	Migration Liquibase

TAB. 10.4 : Récapitulatif des 37 fichiers créés

10.3.2 Fichiers Modifiés

Fichier	Modification
db/changelog/db.changelog-master.yaml	Ajout de l'entrée 010-operations-tables.sql
application/service/FleetManagerService.java	Enrichissement des KPIs avec les données opérationnelles
infrastructure/adapters/inbound/rest/FleetManagerController.java	Enrichissement du Controller

TAB. 10.5 : Fichiers modifiés lors de l'intégration

Chapitre 11

Décisions Techniques et Bilan

11.1 Décisions Techniques Majeures

11.1.1 Pas de Records pour les Entités Mutables

Les entités `Maintenance`, `Incident` et `FuelRecharge` sont des classes Java classiques (pas des records), contrairement aux modèles existants (`Vehicle`, `Driver`, `Fleet`, `Trip` qui sont des records immuables). Cette décision est justifiée par la nature de ces entités :

- `Incident` implémente une machine à états avec des méthodes qui modifient son état (`resolve()`, `close()`)
- `Maintenance` dispose de méthodes métier qui modifient ses champs (`addReport()`, `updateCost()`)
- Les records Java sont immuables par nature, incompatibles avec ces patterns

11.1.2 Fire & Forget pour les Événements

La publication des événements Kafka est réalisée en mode *fire & forget* via `.subscribe()` :

Listing 11.1 : Pattern fire and forget pour les événements

```
1 .doOnSuccess(saved ->
2     operationEventPort.publishMaintenanceCreated(event)
3     .subscribe(
4         null,
5         e -> log.warn("Echec publication evenement: {}", e.
6             getMessage())
7     )
8 )
```

Cette approche garantit que l'échec de publication d'un événement Kafka ne fait pas échouer la création de l'opération. La traçabilité est assurée par les logs.

11.1.3 Synchronisation Inter-Modules via Repositories Directs

Les synchronisations inter-modules (`maintenance_parameters`, `operational_parameters`) sont réalisées en injectant directement les repositories R2DBC dans les services, plutôt que de créer de nouveaux ports. Cette décision évite la sur-ingénierie pour des cas d'usage simples déjà couverts par les repositories existants.

11.1.4 Enums en VARCHAR

Les enums sont stockés en VARCHAR plutôt qu'en types ENUM PostgreSQL natifs. Cette décision est cohérente avec le reste du projet et évite les problèmes de conversion avec R2DBC, qui ne supporte pas nativement les types ENUM PostgreSQL sans convertisseurs complexes.

11.1.5 Coordonnées en Deux Colonnes

Les coordonnées GPS sont stockées en deux colonnes séparées (`longitude`, `latitude`) plutôt qu'en type PostGIS POINT. Cette décision évite la dépendance à l'extension PostGIS dans les entités R2DBC et simplifie la conversion dans les adapters.

11.2 Respect des Règles Architecturales

Règle	Respectée	Vérification
Le domaine ne dépend de rien	Oui	Aucune annotation Spring/JPA dans <code>domain/</code>
Les IDs sont des UUID	Oui	Tous les IDs sont UUID
Tout est réactif	Oui	Mono/Flux dans tous les ports et services
Invariants dans le constructeur	Oui	Validation dans les constructeurs des entités
Commands en records imbriqués	Oui	Records dans les interfaces Use Case
Exceptions typées	Oui	<code>OperationException</code> avec codes OPR_XXX
Adapters sans logique métier	Oui	Adapters = conversion + délégation uniquement

Règle	Respectée	Vérification
Tables dans le schéma <code>fleet</code>	Oui	Toutes les nouvelles tables dans <code>fleet.*</code>

TAB. 11.1 : Vérification du respect des règles architecturales

11.3 Bilan Quantitatif

Indicateur	Valeur
Fichiers Java créés	36
Fichiers Java modifiés	2
Fichiers SQL créés	1
Fichiers YAML modifiés	1
Nouveaux endpoints REST	28
Nouvelles tables PostgreSQL	3
Nouveaux index de base de données	9
Phases d'implémentation	6
Erreurs de compilation	0

TAB. 11.2 : Bilan quantitatif des apports

11.4 Intégration avec les Modules Existants

Le module Opérations Terrain s'intègre naturellement avec les modules existants :

- **Module Véhicule** : Vérification de l'existence du véhicule avant toute opération, mise à jour des paramètres de maintenance et du niveau carburant
- **Module Conducteur** : Vérification optionnelle de l'existence du conducteur, dénormalisation du nom pour l'affichage
- **Module Notification** : Publication d'événements Kafka consommés par le module de notification pour alerter le gestionnaire
- **Module Fleet Manager** : Enrichissement des KPIs avec les données opérationnelles (coûts incidents, coûts maintenances, consommation carburant)

Conclusion Générale

Synthèse des Travaux

Ce rapport a présenté l'architecture et l'évolution du backend de la plateforme Fleet-Man, un système de gestion de flotte de véhicules développé en Java 21 avec Spring Boot 3.2.6 et une architecture hexagonale réactive.

La Partie I a décrit l'architecture existante, caractérisée par :

- Une architecture hexagonale (Ports & Adapters) garantissant la séparation stricte des responsabilités
- Un paradigme entièrement réactif (WebFlux, R2DBC, Reactor Kafka) pour des performances optimales
- Une sécurité JWT déléguée à un service d'authentification centralisé
- Des modules fonctionnels couvrant la gestion des véhicules, conducteurs, flottes, trajets et géorepérage
- Une infrastructure robuste avec PostgreSQL, Redis, Kafka et Liquibase

La Partie II a présenté les apports réalisés dans le cadre de ce projet de fin d'études :

- L'intégration du module *Opérations Terrain* avec trois agrégats métier (Maintenance, Incident, FuelRecharge)
- La création de 36 fichiers Java couvrant toutes les couches de l'architecture hexagonale
- L'implémentation de 28 nouveaux endpoints REST documentés Swagger
- La création de 3 nouvelles tables PostgreSQL avec 9 index stratégiques
- Des synchronisations inter-modules enrichissant les données des véhicules et les KPIs des gestionnaires

Apport Pédagogique

Ce projet a permis de mettre en pratique des concepts avancés de génie logiciel :

- **Domain-Driven Design (DDD)** : Identification des agrégats, value objects, invariants métier, bounded contexts
- **Architecture Hexagonale** : Isolation du domaine, ports et adapters, règle de dépendance
- **Programmation Réactive** : Mono/Flux, opérateurs de transformation, gestion des erreurs réactives
- **Patterns d'Intégration** : Anti-Corruption Layer, Event-Driven Architecture, CQRS partiel
- **Qualité Logicielle** : Validation des invariants, gestion des exceptions typées, documentation API

Perspectives d'Évolution

Plusieurs axes d'amélioration sont envisageables pour les prochaines itérations :

1. **Tests automatisés** : Ajout de tests unitaires pour les entités de domaine et de tests d'intégration pour les adapters
2. **Pagination** : Implémentation de la pagination sur les endpoints de listing pour les grandes flottes
3. **Rapports PDF** : Génération automatique de rapports d'incidents et de maintenances
4. **Tableau de bord temps réel** : Intégration WebSocket pour les alertes d'incidents critiques
5. **Mode hors-ligne** : Synchronisation différée pour les conducteurs en zone sans réseau
6. **Intelligence artificielle** : Prédiction des maintenances préventives basée sur l'historique

Conclusion

Le module Opérations Terrain apporte une valeur métier significative à la plateforme FleetMan en comblant un manque fonctionnel majeur. Son intégration respecte scrupuleusement l'architecture hexagonale réactive existante, garantissant la cohérence et la maintenabilité du système. L'ensemble du code compile sans erreur et est prêt pour les tests d'intégration.

Annexe A

Glossaire Technique

Terme	Définition
Agrégat	Groupe d'objets du domaine traités comme une unité cohérente, avec une racine d'agrégat
Architecture Hexagonale	Pattern architectural isolant le domaine métier des dépendances techniques via des ports et adapters
Bounded Context	Limite explicite dans laquelle un modèle de domaine particulier s'applique
DDD	Domain-Driven Design — approche de conception centrée sur le domaine métier
Flux	Type réactif représentant un flux de 0 à N éléments (Project Reactor)
Geofencing	Technique de délimitation de zones géographiques virtuelles avec déclenchement d'alertes
Invariant	Règle métier qui doit toujours être vraie pour un objet du domaine
Liquibase	Outil de gestion des migrations de schéma de base de données
Mono	Type réactif représentant 0 ou 1 résultat (Project Reactor)
Port	Interface définissant un contrat entre le domaine et l'infrastructure
R2DBC	Reactive Relational Database Connectivity — API réactive pour les bases de données relationnelles
Record Java Value Object	Type de classe immuable introduit en Java 16 Objet du domaine défini par ses valeurs, sans identité propre
WebFlux	Framework web réactif de Spring, basé sur Project Reactor

Terme	Définition
-------	------------

TAB. A.1 : Glossaire des termes techniques

Annexe B

Récapitulatif des Endpoints REST

Tableau Complet des Endpoints

Méthode	URL	Module	HTTP
POST	/api/v1/auth/login	Auth	200
POST	/api/v1/auth/register	Auth	200
POST	/api/v1/auth/refresh	Auth	200
POST	/api/v1/vehicles	Véhicule	201
GET	/api/v1/vehicles	Véhicule	200
GET	/api/v1/vehicles/{id}	Véhicule	200
PATCH	/api/v1/vehicles/{id}	Véhicule	200
DELETE	/api/v1/vehicles/{id}	Véhicule	204
GET	/api/v1/vehicles/lookup/{resource}	Véhicule	200
POST	/api/v1/fleets/{id}/drivers/register	Conducteur	201
GET	/api/v1/drivers	Conducteur	200
POST	/api/v1/drivers/{id}/assign-vehicle	Conducteur	200
POST	/api/v1/trips/start	Trajet	201
POST	/api/v1/trips/{id}/telemetry	Trajet	200
POST	/api/v1/trips/{id}/end	Trajet	200
GET	/api/v1/trips	Trajet	200
POST	/api/v1/geofence/zones	Geofence	201
GET	/api/v1/geofence/my-zones	Geofence	200
DELETE	/api/v1/geofence/{type}/{id}	Geofence	204
POST	/api/v1/operations/maintenances	Maintenance	201
GET	/api/v1/operations/maintenances	Maintenance	200
GET	/api/v1/operations/maintenances/{id}	Maintenance	200
PUT	/api/v1/operations/maintenances/{id}	Maintenance	200
DELETE	/api/v1/operations/maintenances/{id}	Maintenance	204
POST	/api/v1/operations/incidents	Incident	201
GET	/api/v1/operations/incidents	Incident	200

Méthode	URL	Module	HTTP
GET	/api/v1/operations/incidents/open	Incident	200
GET	/api/v1/operations/incidents/filters	Incident	200
PATCH	/api/v1/operations/incidents/{id}	Incident	200
DELETE	/api/v1/operations/incidents/{id}	Incident	204
POST	/api/v1/operations/fuel-recharges	Carburant	201
GET	/api/v1/operations/fuel-recharges	Carburant	200
DELETE	/api/v1/operations/fuel-recharges/{id}	Carburant	204

TAB. B.1 : Sélection des endpoints REST principaux (les modules en gras sont les apports)